1. **Implement Linux memory management commands: Overview, free, vmstat, Memory Allocation System Calls (malloc, sbrk). Analyse how memory is allocated and utilized by running processes.**

Objective:

To provide students with an in-depth understanding of Linux virtual memory management architecture, system-wide memory telemetry commands (free, vmstat), user-space memory management (malloc), and low-level kernel system calls (brk, sbrk, mmap). Students will learn how Linux allocates, tracks, and isolates virtual address spaces across executing processes.

Learning Outcomes

By the end of this lab, students will be able to:

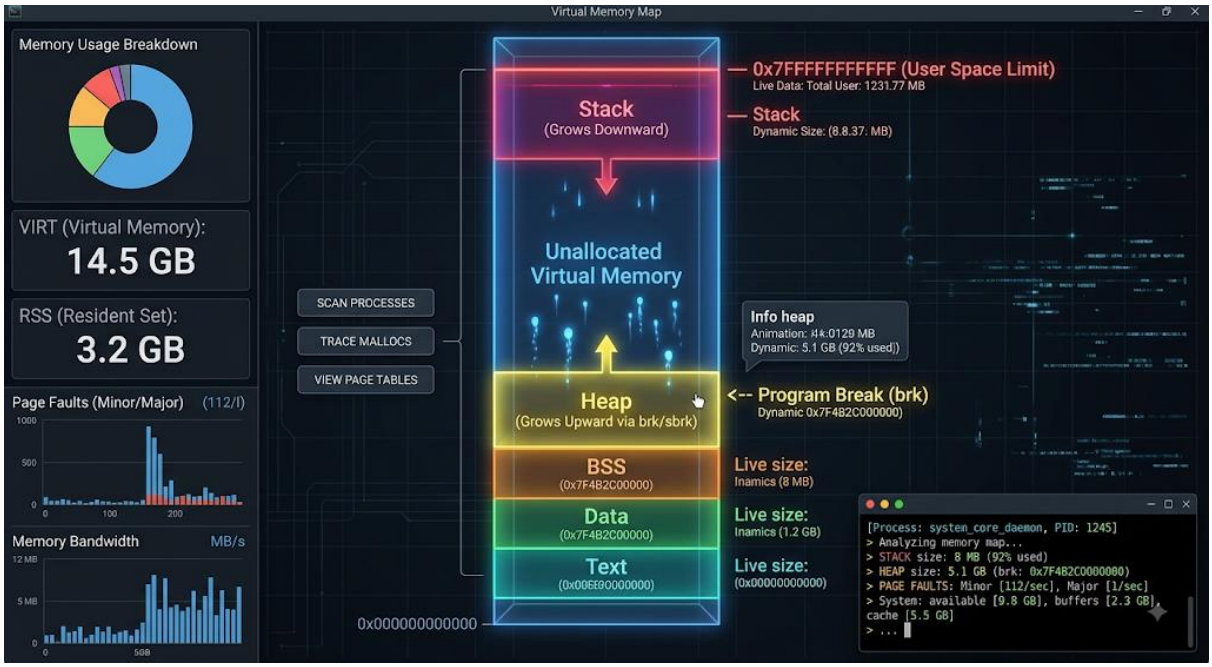
- **Interpret** system-wide memory metrics (total, available, used, buffers, page cache, swap) using free and vmstat.
- **Differentiate** between Virtual Memory Size (**VIRT**), Resident Set Size (**RSS**), and Shared Memory (**SHR**) in process execution environments.
- **Explain** how malloc() uses brk() / sbrk() for small heap allocations and mmap() for large anonymous memory mappings.
- **Inspect** process address space maps using /proc/[pid]/maps and pmap.

Procedure:

Introduction:
Linux implements a **Virtual Memory Architecture** based on dynamic paging, isolating each process inside a contiguous virtual address space.

- **Pages and Page Tables:** Virtual memory is divided into fixed-size chunks called **pages** (typically 4 KB). The Memory Management Unit (**MMU**) translates virtual addresses to physical RAM frames using page tables.
- **Lazy Allocation (Demand Paging):** Calling memory allocation routines does not immediately allocate physical RAM. The OS grants virtual memory addresses; physical pages are allocated only when the application reads or writes to that address, triggering a **minor page fault**.
- **Page Cache & Buffers:** Unused physical memory is automatically utilized by Linux to cache disk blocks (Page Cache) to accelerate file I/O operations. This memory is instantly reclaimed if applications demand it.

	Page Cache	Buffers
Key task	Actual file data (contents of files you read or write)	Raw block-device data and metadata (inodes, directory entries, superblocks, partition tables)
Level	Filesystem level, works with files	Block device level, works with disk blocks
Example	Reading a large log file, so the next read comes from RAM	Reading directory structure or writing to a disk block directly



free Command

Displays the total amount of free and used physical memory and swap space in the system, as well as the buffers and caches used by the kernel.

free [OPTIONS]

Flag	Purpose
-h	Display amounts in human-readable units (M = Megabytes, G = Gigabytes)

Flag	Purpose
-m	Display memory in Megabytes
-s <sec>	Continuously refresh output every <sec> seconds
-t	Display a total line for RAM + Swap combined

Understanding free -h Output Fields

- **total**: Total installed RAM available to the OS.
- **used**: Memory currently consumed by processes (total - free - buff/cache).
- **free**: Unallocated memory entirely unused by the kernel.
- **shared**: Memory used mainly by tmpfs (shared memory / IPC).
- **buffer/cache**: Combined memory used by kernel buffers (metadata) and page cache (file caching).
- **available**: Estimated RAM available for starting new applications without swapping.

vmstat Command

Reports virtual memory statistics, process activity, block I/O, paging, trap execution, and CPU activity.

vmstat [INTERVAL] [COUNT]

Sample: Print 5 updates at 1-second intervals

vmstat 1 5

Field Breakdown

Section	Column	Description
Procs	r	Runnable processes waiting for CPU time
	b	Processes in uninterruptible sleep (I/O blocked)

Section	Column	Description
Memory	swpd	Virtual memory used (KB)
	free	Idle memory (KB)
	buff	Memory used as buffers (KB)
	cache	Memory used as cache (KB)
Swap	si	Memory swapped **in** from disk per second
	so	Memory swapped **out** to disk per second
IO	bi	Blocks received from a block device (Read KB/s)
	bo	Blocks sent to a block device (Write KB/s)
System	in	Interrupts per second (including clock)
	cs	Context switches per second
CPU	us, sy, id, wa	User time, System (kernel) time, Idle time, I/O Wait time (%)

Memory Allocation: malloc vs System Calls (brk, sbrk, mmap)

User applications rarely request memory directly from the Linux kernel. Instead, they use C library allocators (glibc malloc).

brk(void *addr) / sbrk(intptr_t increment):

Adjusts the location of the **program break** (the end address of the process heap).

Used by malloc() for small memory allocations (typically < 128 KB by default in glibc).

mmap(..., MAP_ANONYMOUS | MAP_PRIVATE, ...):

Maps a new block of pages directly into the process's address space outside the traditional heap segment.

Used by malloc() for large allocations (typically ≥ 128 KB). When freed with free(), the memory is immediately returned to the OS kernel.

malloc(size_t size):

User-space library wrapper that manages pre-allocated memory chunks in pool arenas to minimize costly system call overhead.

Baseline Memory Inspection: Baseline Telemetry.

Run system-wide memory metrics to establish baseline memory usage before running custom programs.

View human-readable memory distribution

free -h

Monitor virtual memory activity (1-second intervals for 3 seconds)

vmstat 1 3

Tracing Heap Expansion with sbrk(): System Call Tracing.

Write and execute a C program that directly invokes sbrk() to track how the program break pointer moves as memory is allocated.

Create sbrk_demo.c:

C

```
#include <stdio.h> // For printf()

#include <unistd.h> // For sbrk()

#include <stdint.h> // For intptr_t

int main() {

   // Retrieve current program break

   void *current_brk = sbrk(0);

   printf("[START] Current Program Break: %p\n", current_brk);
```

```
    // Expand heap by 128 KB (131072 bytes)
    intptr_t increment = 131072;
    void *previous_brk = sbrk(increment);
    void *new_brk = sbrk(0);

    printf("[AFTER sbrk(+128KB)]\n");
    printf("  Previous Break : %p\n", previous_brk);
    printf("  New Break      : %p\n", new_brk);
    printf("  Address Offset : %ld bytes\n", (char *)new_brk - (char *)previous_brk);

    // Contract heap back to original break position
    sbrk(-increment);
    printf("[RESTORED] Program Break: %p\n", sbrk(0));

    return 0;
}
```

Observing malloc Thresholds (Heap vs. mmap): Allocation Thresholds.

Analyze how malloc() uses brk for small chunks and mmap for large chunks using the strace utility.

```
#include <stdio.h>   // For printf() and perror()
#include <stdlib.h>  // For malloc(), free(), size_t, and NULL
#include <unistd.h>  // For getpid()
#include <malloc.h>  // For mallopt() and M_MMAP_THRESHOLD

int main() {
```

```
// Explicitly set the mmap threshold to 128 KB (131072 bytes) for deterministic behavior
mallopt(M_MMAP_THRESHOLD, 128 * 1024);

printf("PID: %d\n\n", getpid());

// --- 1. Small Allocation (< 128 KB) ---
size_t small_size = 10 * 1024; // 10 KB
printf("1. Requesting Small Allocation (%zu bytes)...\n", small_size);
void *small_ptr = malloc(small_size);
if (small_ptr == NULL) {
   perror("Small allocation failed");
   return 1;
}
printf("   Address: %p (Allocated via Heap/brk)\n\n", small_ptr);

// --- 2. Large Allocation (>= 128 KB) ---
size_t large_size = 500 * 1024; // 500 KB
printf("2. Requesting Large Allocation (%zu bytes)...\n", large_size);
void *large_ptr = malloc(large_size);
if (large_ptr == NULL) {
   perror("Large allocation failed");
   free(small_ptr);
   return 1;
}
printf("   Address: %p (Allocated via mmap)\n\n", large_ptr);
```

```
    // --- 3. Freeing Memory ---
    printf("3. Freeing Small Allocation...\n");
    free(small_ptr); // Returned to malloc pool (brk unchanged)

    printf("4. Freeing Large Allocation...\n");
    free(large_ptr); // Returned immediately to OS via munmap()

    return 0;
}
```

Analyze physical memory usage (**RSS**) vs. virtual memory allocation (**VIRT**) using /proc/[pid]/maps and pmap.

Program:

```
#include <stdio.h>   // For printf(), getchar(), and perror()
#include <stdlib.h>  // For malloc(), free(), and size_t
#include <string.h>  // For memset()
#include <unistd.h>  // For getpid()

int main() {
    printf("Process PID: %d\n", getpid());
    printf("Press ENTER to allocate virtual memory...");
    getchar();

    size_t size = 100 * 1024 * 1024; // 100 MB
    char *buffer = (char *)malloc(size);
    if (buffer == NULL) {
```

```
        perror("Memory allocation failed");
        return 1;
    }

    printf("[MALLOC COMPLETE] Allocated 100MB Virtual Memory.\n");
    printf("Press ENTER to touch physical memory (trigger page faults)...");
    getchar();

    // Write data to trigger physical page allocation (Demand Paging)
    memset(buffer, 'A', size);
    printf("[TOUCH COMPLETE] Written to 100MB Physical Memory.\n");
    printf("Press ENTER to exit...");
    getchar();

    free(buffer);
    return 0;
}
```